CLOUD-05: Azure Integration

> **Series:** CLOUD — Cloud Provider Integrations | **Notebook:** 5 of 8 | **Created:** March 2026 | **Last Updated:** 07/20/2026

Overview

This notebook covers Dynatrace's integration with Microsoft Azure. You will learn how to configure Azure Monitor metrics ingestion, which Azure services are supported, authentication via Azure AD (Entra ID), how resource group organization maps to Dynatrace monitoring, and how to query Azure entities and metrics with DQL.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
Permissions	`metrics.read`, `entities.read`, `ReadConfig`, `WriteConfig`
Azure Subscription	With Entra ID (Azure AD) app registration for Dynatrace
Connection	Azure Native Dynatrace Service (recommended) or Clouds app or Environment ActiveGate (classic)
Prior Knowledge	CLOUD-01 fundamentals

1. Azure Integration Architecture

Dynatrace offers multiple integration paths for Azure, with the **Azure Native Dynatrace Service** being the most streamlined for new deployments.

Connection Methods

Method	Mechanism	Best For
Azure Native Dynatrace Service	Fully managed via Azure Marketplace — no ActiveGate needed	New deployments, Azure-native teams
Clouds app (direct connection)	Connect via Entra ID app registration from Dynatrace SaaS	Existing Dynatrace SaaS environments
Classic ActiveGate polling	ActiveGate queries Azure Monitor API every 5 min	Managed deployments, legacy setups
Azure Event Hub	Streams metrics/logs via Event Hub	Near real-time log ingestion

Azure Native Dynatrace Service

The Azure Native Dynatrace Service is available through the **Azure Marketplace** and provides:

- **Zero-infrastructure setup** — no ActiveGate or additional VMs required
- **Unified billing** through Azure Marketplace
- **One-click log forwarding** — subscription activity logs and resource logs forwarded directly
- **OneAgent auto-deployment** as an extension on VMs and App Services
- **SSO integration** with Entra ID

Data Flow (Modern)

```
Azure Resources → Azure Monitor → Dynatrace SaaS (direct connection)
                → Diagnostic Settings → Dynatrace (logs via Event Hub or direct)
```

Supported Azure Regions

Dynatrace supports all Azure public regions, Azure Government, and Azure China (with separate configuration).

Release Radar (April 2026): Native Azure Experience in Clouds is GA

The enhanced **Clouds app** experience — already available for AWS — is now **generally available for Microsoft Azure**, bringing Azure to parity. For net-new Azure onboarding, this is the recommended path; the connection-mechanism table above still describes what runs underneath.

What the native Azure experience adds:

Capability	What it gives you
Unified resource view	Metrics, logs, metadata, and topology for Azure subscriptions in one place, alongside AWS
Opinionated insights & dashboards	Pre-built dashboards and investigations over enriched Azure telemetry to shorten root-cause analysis

Capability	What it gives you
Pre-configured health alerts	Health alerts created and managed directly in the Clouds app, with drill-down, search, and filtering scoped to Azure resources
Broad metric coverage	Any Azure Monitor native platform metric across services, queryable through DQL
Topology inventory	Periodic scanning enriched with native metadata (tags, subscription IDs), fully queryable via DQL
Onboarding & lifecycle management	Centralized provisioning that converts Azure subscriptions into native Dynatrace connections, reducing operational overhead

> **Note:** Azure resource topology and metadata surface through the modern Smartscape model. The `dt.entity.azure_*` entity types in the queries below remain valid for entity lookups; for new topology-traversal queries prefer `dt.smartscape.*` and `smartscapeNodes`.

2. Authentication Setup

For non-native integrations, Azure uses an Entra ID (Azure AD) app registration. The Azure Native Dynatrace Service handles authentication automatically.

Setup Steps

1. **Register an application** in Entra ID (Azure Active Directory)
2. **Create a client secret** (or configure certificate-based auth)
3. **Assign the Reader role** at the subscription or management group level
4. **Configure in Dynatrace** with Tenant ID, Client ID, and Client Secret

Required Azure Permissions

Scope	Role	Purpose
Subscription	**Reader**	Access resource metadata and metrics

Scope	Role	Purpose
Subscription	**Monitoring Reader**	Access Azure Monitor data (alternative to Reader)
Management Group	**Reader**	Multi-subscription monitoring

Authentication Methods Comparison

Method	Security Level	Rotation	Recommended
Client secret	Medium	Manual (1-2 year expiry)	Development
Certificate	High	Certificate lifecycle	Production
Managed identity	Highest	Automatic	ActiveGate on Azure VM

> **Best Practice:** Use the **Azure Native Dynatrace Service** for the simplest setup. If using classic integration with an ActiveGate on an Azure VM, use **managed identity** to eliminate credential management.

3. Supported Azure Services

Dynatrace monitors 50+ Azure services. Key services:

Category	Services
Compute	Virtual Machines, VM Scale Sets, App Service, Azure Functions, Container Instances
Containers	AKS (Azure Kubernetes Service), Container Apps
Database	SQL Database, Cosmos DB, Database for PostgreSQL/MySQL, Cache for Redis
Storage	Blob Storage, File Storage, Queue Storage, Data Lake

Category	Services
Networking	Load Balancer, Application Gateway, Front Door, Traffic Manager, VPN Gateway
Integration	Service Bus, Event Hub, Event Grid, Logic Apps, API Management
AI/ML	Cognitive Services, Machine Learning, OpenAI Service

Entity Type Mapping

Azure Resource	Dynatrace Entity Type
Virtual Machine	`dt.entity.azure_vm`
Web App / App Service	`dt.entity.azure_web_app`
SQL Database	`dt.entity.azure_sql_database`
Cosmos DB	`dt.entity.azure_cosmos_db`
Load Balancer	`dt.entity.azure_load_balancer`
IoT Hub	`dt.entity.azure_iot_hub`

4. Querying Azure Entities

List Azure Virtual Machines

```
// List all monitored Azure VMs
fetch dt.entity.azure_vm
| fieldsKeep id, entity.name, tags
| sort entity.name asc
| limit 20
```

Count Azure Resources by Type

```
// Count Azure VMs and Web Apps
fetch dt.entity.azure_vm
| summarize resource_count = count()
| fieldsAdd provider = "Azure", resource_type = "Virtual Machine"
| append [
    fetch dt.entity.azure_web_app
    | summarize resource_count = count()
    | fieldsAdd provider = "Azure", resource_type = "Web App"
  ]
| append [
    fetch dt.entity.azure_sql_database
    | summarize resource_count = count()
    | fieldsAdd provider = "Azure", resource_type = "SQL Database"
  ]
| sort resource_count desc
```

List Azure Web Apps

```
// List monitored Azure App Service instances
fetch dt.entity.azure_web_app
| fieldsKeep id, entity.name, tags
| sort entity.name asc
| limit 20
```

5. Azure Metrics with DQL

Azure Monitor metrics are ingested into Dynatrace under the `cloud.azure.*` namespace.

Azure VM CPU Usage

```
// Azure VM CPU percentage over the last 6 hours
timeseries avgCpu = avg(dt.host.cpu.usage), from:-6h, by:{dt.entity.host}
| fieldsAdd avgCpuValue = arrayAvg(avgCpu)
| sort avgCpuValue desc
| limit 10
```

Azure VM Memory Usage

```
// Host memory usage for Azure VMs over the last 6 hours
timeseries avgMem = avg(dt.host.memory.usage), from:-6h, by:{dt.entity.host}
| fieldsAdd avgMemValue = arrayAvg(avgMem)
| sort avgMemValue desc
| limit 10
```

6. AKS Monitoring

Azure Kubernetes Service (AKS) monitoring follows similar patterns to EKS (see CLOUD-03) with Azure-specific considerations.

AKS vs EKS Monitoring Differences

Aspect	AKS	EKS
Managed control plane	Yes (free tier)	Yes
Node agent	DynaKube Operator	DynaKube Operator
Native monitoring	Azure Monitor for containers	CloudWatch Container Insights
Network plugin	Azure CNI / Kubenet	VPC CNI
Serverless nodes	Virtual Nodes (ACI)	Fargate
Authentication	Entra ID (Azure AD) integration	IAM/IRSA

AKS Node Metrics

```
// AKS container CPU usage by namespace over the last hour
timeseries containerCpu = avg(dt.kubernetes.container.cpu_usage), from:-1h,
by:{k8s.namespace.name}
| fieldsAdd avgCpu = arrayAvg(containerCpu)
| sort avgCpu desc
| limit 10
```

AKS Pod Memory

```
// Container memory by namespace over the last hour
timeseries containerMem = avg(dt.kubernetes.container.memory_working_set),
from:-1h, by:{k8s.namespace.name}
| fieldsAdd avgMemBytes = arrayAvg(containerMem)
| fieldsAdd avgMemMB = avgMemBytes / 1048576.0
| sort avgMemMB desc
| limit 10
```

7. Resource Group Mapping

Azure organizes resources into **resource groups**, which serve as logical containers. Dynatrace captures this organization for filtering and governance.

Mapping Strategy

Azure Construct	Dynatrace Equivalent	Purpose
Subscription	Tag / Management Zone	Top-level organizational boundary
Resource Group	Tag / Management Zone	Logical grouping for team/application
Tags	Dynatrace tags	Flexible metadata (environment, cost-center, team)

Best Practices

- **Map resource groups to Management Zones** (or Segments in Grail) for access control
- **Propagate Azure tags** to Dynatrace for consistent filtering
- **Use naming conventions** that include subscription, environment, and application: `rg-myapp-prod-eastus`
- **Create alerting profiles** per resource group to route alerts to the right team

8. Summary and Next Steps

Key Takeaways

- Azure integration uses **Entra ID app registration** with Reader role for authentication
- **Managed identity** is the most secure option when ActiveGate runs on an Azure VM
- **AKS monitoring** uses the same DynaKube Operator as EKS with Azure-specific networking considerations
- **Resource groups and Azure tags** should map to Dynatrace Management Zones/Segments for governance

Next Steps

- **CLOUD-06: GCP Integration** — Google Cloud monitoring setup
- **CLOUD-07: CloudWatch Log Ingestion** — Log forwarding patterns (applicable to Azure via Event Hub)
- **CLOUD-08: Multi-Cloud Patterns** — Unified monitoring across Azure and other providers

---